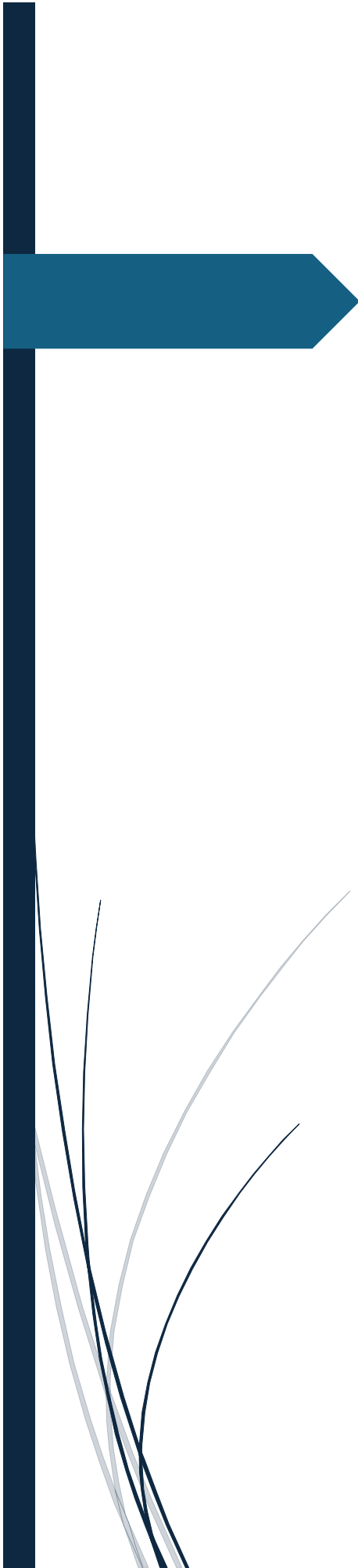




BI para análisis integral de ventas, inventario y rentabilidad

Valentina Benitez, Darieth Sánchez, Samuel Villa
UNIVERSIDAD EAFIT, BASES DE DATOS AVANZADAS

Tabla de Contenido

Base de datos OLTP	2
Volumen de datos	2
Restricciones	2
Base de datos STAGING	4
Transformaciones	4
Campos derivados.....	6
Data Warehouse	7
Diagrama Estrella	7
Justificaciones.....	8
USO DE IA.....	11

1. Base de datos OLTP

1.1. Volumen de datos

Tabla	Registros
Categories	10
SalesChannels	5
Stores	10
Suppliers	30
Customers	1000
Salespersons	20
Products	200
Campaigns	5
Sales	50000
SalesDetails	150000
DailyInventory	730000
Purchases	5000
PurchaseDetails	20000
Returns	3000
SalesTargets	1200

1.2. Restricciones

Tabla	Restricciones
Categories	PK: CategoryID NOT NULL: CategoryName, IsActive, CreatedDate DEFAULT: IsActive=1, CreatedDate=GETDATE()
SalesChannels	PK: SalesChannelID NOT NULL: ChannelName
Stores	PK: StoreID NOT NULL: StoreName, City, Region, IsActive DEFAULT: IsActive=1
Suppliers	PK: SupplierID NOT NULL: SupplierName, TaxID, City, StateProvince, IsActive DEFAULT: IsActive=1
Customers	PK: CustomerID CHECK: Gender IN ('Male','Female','Other') NOT NULL: FullName, DocumentNumber, RegistrationDate

	DEFAULT: RegistrationDate=GETDATE()
Salespersons	PK: SalespersonID FK: StoreID → Stores(StoreID) NOT NULL: StoreID, FullName, DocumentNumber, IsActive DEFAULT: IsActive=1
Products	PK: ProductID FK: CategoryID → Categories(CategoryID) FK: SupplierID → Suppliers(SupplierID) CHECK: SalePrice > 0, UnitCost > 0 NOT NULL: CategoryID, SupplierID, ProductName, SKU, SalePrice, UnitCost, IsActive DEFAULT: IsActive=1
Campaigns	PK: CampaignID NOT NULL: CampaignName
Sales	PK: SaleID FK: CustomerID → Customers(CustomerID) FK: StoreID → Stores(StoreID) FK: SalespersonID → Salespersons(SalespersonID) FK: SalesChannelID → SalesChannels(SalesChannelID) FK: CampaignID → Campaigns(CampaignID) NOT NULL: CustomerID, StoreID, SalespersonID, SalesChannelID, SaleDate
SalesDetails	PK: SaleDetailID FK: SaleID → Sales(SaleID) FK: ProductID → Products(ProductID) CHECK: Quantity > 0 NOT NULL: SaleID, ProductID, Quantity, UnitPrice, UnitCost, TotalAmount DEFAULT: DiscountAmount=0
DailyInventory	PK: InventoryID FK: ProductID → Products(ProductID) FK: StoreID → Stores(StoreID) NOT NULL: ProductID, StoreID, InventoryDate, InitialStock, StockIn, StockOut, FinalStock
Purchases	PK: PurchaseID

	FK: SupplierID → Suppliers(SupplierID) FK: StoreID → Stores(StoreID) NOT NULL: SupplierID, StoreID, PurchaseDate
PurchaseDetails	PK: PurchaseDetailID FK: PurchaseID → Purchases(PurchaseID) FK: ProductID → Products(ProductID) NOT NULL: PurchaseID, ProductID, Quantity, UnitCost, TotalAmount
Returns	PK: ReturnID FK: SaleID → Sales(SaleID) FK: ProductID → Products(ProductID) NOT NULL: SaleID, ProductID, ReturnDate, Quantity
SalesTargets	PK: TargetID FK: StoreID → Stores(StoreID) FK: CategoryID → Categories(CategoryID) FK: SalespersonID → Salespersons(SalespersonID) NOT NULL: StoreID, CategoryID, YearNumber, MonthNumber, TargetAmount

2. Base de datos STAGING

2.1. Transformaciones

Tabla	Campos derivado
stg_Categories	Limpieza de espacios en nombres y descripciones Normalización a mayúsculas en categorías Eliminación de duplicados mediante ROW_NUMBER()
stg_SalesChannels	Normalización de nombres de canales Eliminación de duplicados mediante ROW_NUMBER()
stg_Stores	Limpieza de nombres y direcciones Estandarización de ciudades y regiones Reemplazo de direcciones y teléfonos nulos

	Conversión de fecha
stg_Suppliers	Limpieza de nombres y NIT Estandarización de ciudades y departamentos Limpieza de teléfonos Normalización y reemplazo de emails nulos
stg_Customers	Limpieza de nombres Normalización de género Estandarización de ciudades y departamentos Limpieza de segmentos Reemplazo de emails y teléfonos nulos Conversión de fecha
stg_Salespersons	Validación de tiendas existentes Limpieza de nombres y documentos Normalización y reemplazo de emails nulos Conversión de fecha Corrección de salarios inválidos
stg_Products	Limpieza de nombres de productos Normalización de marcas y SKU Reemplazo de marcas nulas
stg_Campaigns	Limpieza de nombres Conversión de fechas Validación de porcentajes de descuento Exclusión de campañas con fechas inválidas
stg_Sales	Validación de campañas nulas Conversión de fecha Normalización de métodos de pago Corrección de montos negativos
stg_SalesDetails	Validación de cantidades Corrección de precios, costos y descuentos negativos Validación de montos totales
stg_DailyInventory	Conversión de fecha Validación de inventario inicial, entradas y salidas Recalculo de inventario final
stg_Purchases	Validación de proveedores y tiendas existentes Conversión de fecha Corrección de montos inválidos

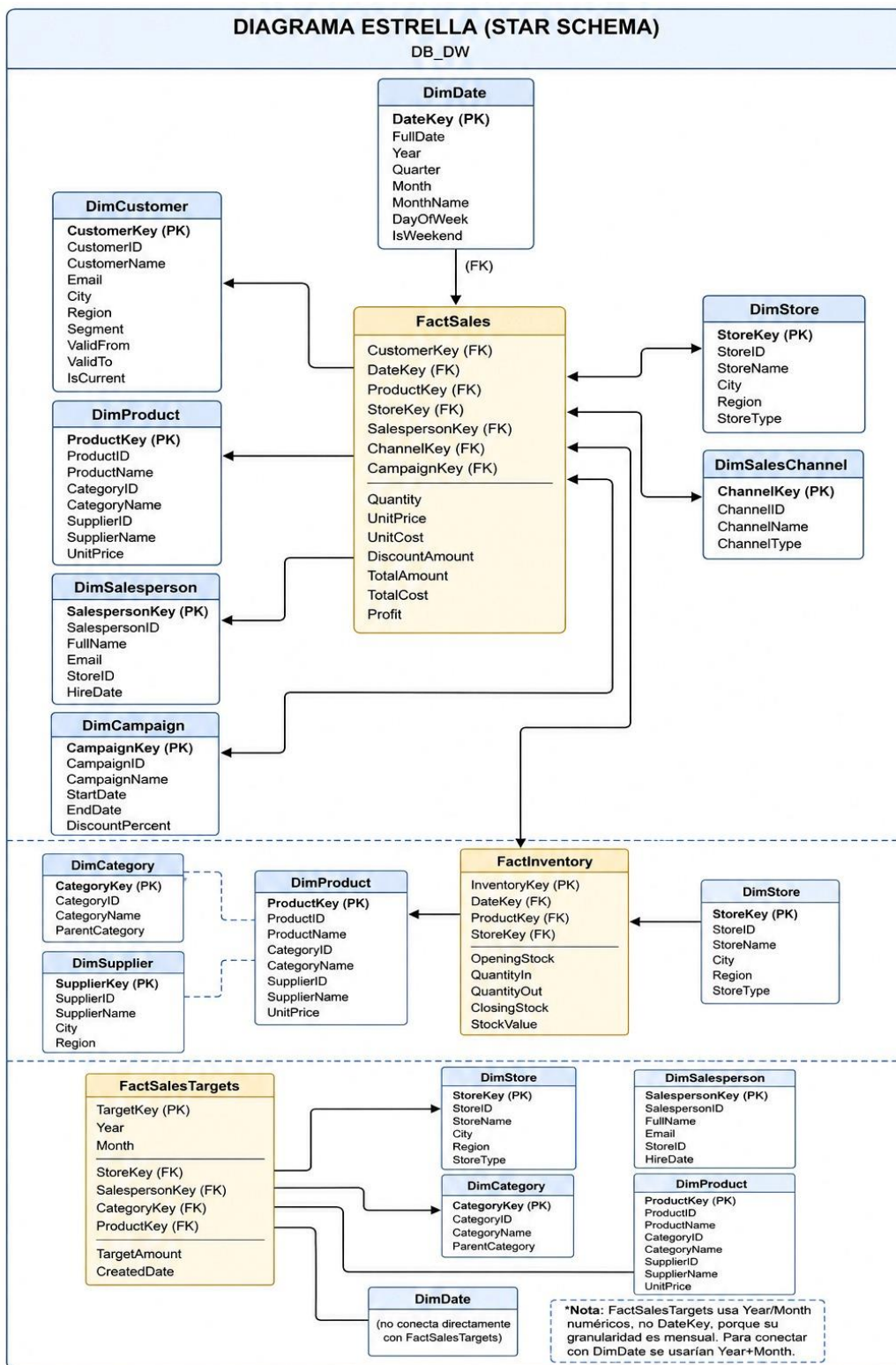
stg_PurchaseDetails	Validación de compras y productos existentes Corrección de cantidades y costos inválidos Recalculo de montos totales
stg>Returns	Validación de ventas y productos existentes Conversión de fecha Corrección de cantidades y montos inválidos Normalización de motivos de devolución
stg_SalesTargets	Asignación automática de vendedores Validación de tiendas y categorías Corrección de años, meses y metas inválidas Distribución equitativa de metas comerciales

2.2. Campos derivados

Tabla	Campos derivado
stg_Categories	Ninguno
stg_SalesChannels	Ninguno
stg_Stores	Ninguno
stg_Suppliers	Ninguno
stg_Customers	Ninguno
stg_Salespersons	Ninguno
stg_Products	ProfitMargin
stg_Campaigns	CampaignDurationDays
stg_Sales	SaleYear, SaleMonth
stg_SalesDetails	NetAmount
stg_DailyInventory	Ninguno
stg_Purchases	PurchaseYear, PurchaseMonth
stg_PurchaseDetails	AverageCost
stg>Returns	Ninguno
stg_SalesTargets	QuarterlyTarget

3. Data Warehouse

3.1. Diagrama Estrella



3.2. Justificaciones

Granularidad de cada tabla de hechos: La granularidad de FactSales es una fila por cada línea de detalle de venta, es decir, cada vez que un cliente compra un producto específico en una transacción se genera un registro único que incluye cantidad, precio, costo y descuento aplicado, lo que permite analizar ventas a nivel más fino posible. La granularidad de FactInventory es una fila por cada combinación de producto, tienda y día, registrando el stock inicial, entradas, salidas y stock final de cada producto en cada sede durante cada jornada, lo que permite rastrear el movimiento de inventario día a día. La granularidad de FactSalesTargets es una fila por cada meta mensual definida por tienda, vendedor y categoría, es decir, cada registro representa el monto de ventas que un vendedor específico debe alcanzar en una tienda determinada durante un mes particular para una categoría de productos concreta.

Medidas aditivas, semi-aditivas y no aditivas: En FactSales, las medidas Quantity, TotalAmount, TotalCost y Profit son completamente aditivas porque tienen sentido sumarse a través de cualquier dimensión como tiempo, cliente, producto o tienda, por ejemplo, sumar las ganancias de todos los productos vendidos en un mes. El UnitPrice es una medida no aditiva porque sumar precios unitarios de diferentes productos no produce ningún significado útil. En FactInventory, el ClosingStock es una medida semi-aditiva porque puede sumarse correctamente a través de productos y tiendas (el inventario total de todos los productos en una tienda), pero no debe sumarse a través del tiempo (sumar inventarios de lunes, martes y miércoles sería duplicar el conteo del mismo producto). El StockValue también es semi-aditivo por la misma razón. En FactSalesTargets, el TargetAmount es completamente aditivo porque se puede sumar por tienda, por vendedor, por categoría o por mes sin ningún problema conceptual.

Problemas de resumibilidad que aparecen: El principal problema de resumibilidad ocurre cuando se intentan agregar medidas de diferentes granularidades dentro de una misma tabla o consulta. Por ejemplo, si en FactSalesTargets existieran algunas metas definidas por tienda (sin especificar vendedor ni categoría) junto con otras metas definidas por vendedor y categoría, al sumar todas las metas para un mes se estarían duplicando valores porque una meta de tienda ya incluye implícitamente a todos los vendedores y categorías. En nuestro modelo evitamos este problema porque todas las metas en FactSalesTargets tienen la misma granularidad específica

por tienda, vendedor y categoría, y cuando una meta no aplica para algún nivel (por ejemplo, meta general de tienda) simplemente no se carga en esta tabla. Otro problema potencial es que en FactSales la medida Profit al sumarse correctamente no presenta problemas de resumibilidad, pero si alguien intentara calcular un margen promedio sumando márgenes porcentuales individuales y dividiendo obtendría un resultado incorrecto, por eso el margen debe calcularse siempre como suma de utilidades dividido suma de ventas.

Por qué inventario no debe sumarse directamente a través del tiempo: El

inventario es una variable de nivel o stock, no un flujo. Cuando se suma el inventario del lunes (10 unidades), más el del martes (8 unidades), más el del miércoles (12 unidades), se obtiene 30 unidades, pero ese número no representa absolutamente nada real porque no hubo 30 unidades acumuladas en ningún momento; simplemente se están contando las mismas unidades físicas varias veces en diferentes días. La forma correcta de analizar inventario a través del tiempo es utilizando promedios (inventario promedio del mes), valores al cierre (inventario final del período), o análisis de tendencias (máximos y mínimos). Por esta razón en nuestro modelo FactInventory declaramos el ClosingStock como medida semi-aditiva y documentamos que no debe sumarse directamente sobre la dimensión de tiempo, sino que en Power BI se deben crear medidas como Inventario Promedio = `AVERAGE(FactInventory[ClosingStock])` o Inventario Final = `CALCULATE(SUM(FactInventory[ClosingStock]), LASTDATE(DimDate[FullDate]))`.

Dimensiones que tienen jerarquías: DimDate tiene una jerarquía temporal natural que permite drill-down desde año hasta trimestre, luego mes y finalmente día, lo que es fundamental para análisis de inteligencia de tiempo como variaciones mensuales o acumulados anuales. DimProduct tiene una jerarquía implícita a través de DimCategory donde cada producto pertenece a una categoría, permitiendo agrupar ventas por categoría y luego desglosar por producto individual. DimStore tiene una jerarquía geográfica donde las tiendas pertenecen a ciudades y las ciudades a regiones, lo que permite análisis desde nivel regional hasta tienda específica. DimCustomer tiene una jerarquía de segmentación donde los clientes se clasifican en CORPORATIVO, PREMIUM o REGULAR, permitiendo analizar comportamiento por segmento y luego por cliente individual. DimSalesperson tiene una relación con DimStore donde cada vendedor pertenece a una tienda, creando una jerarquía de tienda-vendedor.

Dimensiones que podrían manejar Slowly Changing Dimensions tipo 2:

DimCustomer es la candidata principal para SCD Tipo 2 porque los clientes pueden cambiar de ciudad, región o segmento con el tiempo; por ejemplo, un cliente que era REGULAR y luego se vuelve PREMIUM debe mantener su historial de compras asociado al segmento anterior para no distorsionar análisis pasados, y sus compras futuras deben asociarse al nuevo segmento, por eso nuestra tabla ya incluye las columnas ValidFrom, ValidTo e IsCurrent preparadas para implementar SCD Tipo 2 cuando se necesite. DimProduct también podría manejar SCD Tipo 2 si su precio de venta cambia frecuentemente, porque para análisis histórico de rentabilidad se necesita saber con qué precio se vendió cada producto en cada momento, aunque actualmente el precio está en la tabla de hechos FactSales como UnitPrice congelado al momento de la venta, lo que es una práctica mejor que modificar la dimensión. DimSalesperson podría necesitar SCD Tipo 2 si un vendedor cambia de tienda, porque sus ventas históricas deben seguir asociadas a la tienda donde trabajaba en ese momento, no a la nueva. DimStore rara vez necesita SCD Tipo 2 porque las tiendas no cambian de ciudad o región, pero si una tienda cambia de nombre o categoría, se podría implementar también.

Diferencia entre la base OLTP y el modelo OLAP construido: La base OLTP (original DB_OLTP) está diseñada para procesar transacciones diarias de forma eficiente, por lo que está altamente normalizada en tercera forma normal con muchas tablas pequeñas como Customers, Products, Sales, SalesDetails, Stores, etc., cada una con sus propias claves primarias y foráneas, y su objetivo principal es insertar, actualizar y eliminar registros rápidamente sin redundancia. En cambio, nuestro modelo OLAP en DB_DW está diseñado específicamente para análisis y consultas de negocio, por lo que utiliza un esquema estrella desnormalizado con pocas tablas grandes de hechos y tablas dimensionales que contienen atributos redundantes pero que permiten responder preguntas complejas como "ventas por categoría y región mes a mes" con consultas simples y rápidas. Otra diferencia clave es que el OLTP guarda datos solo del estado actual (no guarda histórico de cambios en clientes o productos), mientras que nuestro OLAP prepara estructuras para manejar cambios lentos con SCD Tipo 2. Finalmente, el OLTP no precalcula métricas agregadas como la utilidad por línea de venta, mientras que nuestro OLAP en FactSales ya tiene columnas precalculadas como Profit y TotalCost para acelerar el análisis en Power BI.

4. USO DE IA

Durante el desarrollo del proyecto se utilizó inteligencia artificial como herramienta de apoyo para acelerar tareas técnicas y operativas relacionadas con el diseño e implementación de la solución de bases de datos y procesos ETL. Su uso se enfocó principalmente en:

- Generación de scripts SQL para creación de tablas, relaciones, inserción de datos sintéticos y consultas.
- Apoyo en el diseño inicial del modelo de datos y definición de estructuras tipo estrella.
- Generación de datos de prueba con inconsistencias controladas para aplicar procesos ETL y transformaciones en la zona STAGING.
- Apoyo en la construcción de procedimientos de limpieza y transformación de datos.
- Sugerencias para validaciones, normalización de datos y creación de campos derivados.

Sin embargo, las decisiones principales del proyecto fueron realizadas y validadas manualmente, especialmente en aspectos como:

- Definición de granularidad de hechos y dimensiones.
- Diseño lógico y relacional de la solución.
- Identificación y corrección de inconsistencias reales en los datos.
- Implementación y validación de procesos ETL.
- Verificación de integridad y consistencia de los datos transformados.

La IA fue utilizada como herramienta de apoyo técnico, manteniendo siempre la comprensión y validación de cada componente implementado dentro del proyecto.